

AKVORADO

Guía de Instalación, Configuración y Ajustes

Plataforma de visibilidad y análisis de tráfico de red basada en NetFlow, sFlow e IPFIX.

Este documento cubre la instalación base de Akvorado, los ajustes de producción aplicados, los problemas que se resolvieron y la implementación de BMP para enriquecimiento de datos BGP.

Detalles del documento

Autor: Ariel Angulo Méndez

Fecha: Mayo 2026

1. ¿Qué es?

2. Requisitos

3. Instalación

4. Configuración

5. Ajustes

6. BMP

7. Exportadores

8. Monitoreo

9. Troubleshooting

1. ¿Qué es Akvorado?

Akvorado es una plataforma open source para capturar, procesar y visualizar flujos de red. Permite visibilidad completa del tráfico: qué hosts se comunican, cuánto consumen, por qué interfaz, hacia qué ASN, y más, todo en tiempo real.

La plataforma está compuesta por cinco servicios que corren en contenedores Docker:

Inlet

Recibe los flujos UDP enviados por los routers (NetFlow, sFlow, IPFIX) y los publica en Kafka para su procesamiento.

Outlet

Consume los mensajes de Kafka, enriquece los flujos con datos de SNMP y BGP/BMP, y los escribe en ClickHouse.

ClickHouse

Base de datos columnar de alto rendimiento donde se almacenan todos los flujos. Permite consultas analíticas rápidas sobre grandes volúmenes de datos.

Kafka

Sistema de mensajería que actúa como buffer entre el inlet y el outlet. Garantiza que ningún flujo se pierda ante picos de tráfico.

Console

Interfaz web donde se visualizan los flujos, se generan gráficas de tráfico y se consulta por ASN, interfaz, país, protocolo, entre otros.

Info: Todos los componentes se orquestan con Docker Compose. No se necesita instalar nada por separado.

2. Requisitos del Sistema

Sistema Operativo

- Ubuntu Server 22.04 LTS o superior
- Acceso con usuario sudo o root
- Conectividad de red con los routers exportadores

Hardware

Componente	Especificación
CPU	8 núcleos o más
RAM	16 GB mínimo — 32 GB recomendado para alto volumen
Disco	200 GB SSD — el consumo depende de la retención configurada
Red	Acceso UDP desde los exportadores al servidor

Software Previo

Herramienta	Versión mínima
Docker Engine	24.x o superior
Docker Compose	v2 (incluido en Docker Engine reciente)
Git	Cualquier versión reciente

Importante: ClickHouse es el componente que más memoria consume. Con volúmenes altos de flujos (70k+ flows/s) se recomienda asignarle al menos 8 GB y dejar margen para los demás servicios.

3. Instalación

El proceso completo toma entre 15 y 20 minutos. Se realiza desde la terminal del servidor.

1

Instalar Docker

Si el servidor no tiene Docker instalado, ejecutar los siguientes comandos.

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker
```

2

Clonar el repositorio

Descargar el código fuente oficial de Akvorado desde GitHub.

```
git clone https://github.com/akvorado/akvorado.git
cd akvorado/docker
```

3

Preparar la configuración

Copiar los archivos de ejemplo incluidos en el repositorio.

```
cp akvorado.yaml.sample akvorado.yaml
cp console.yaml.sample console.yaml
```

Recomendado: En este punto ya se puede levantar el servicio con la configuración base para verificar que funcione antes de personalizar.

4

Levantar los servicios

Docker Compose descarga las imágenes y levanta todos los contenedores.

```
docker compose up -d
```

Verificar que todos los contenedores estén corriendo y sanos:

```
docker ps
```

Todos los servicios deben aparecer con estado (healthy). El proceso puede tomar 1 a 2 minutos la primera vez.

5

Acceder a la consola web

Una vez levantado, la interfaz web está disponible en el servidor.

```
http://<IP-DEL-SERVIDOR>:8081
```

4. Configuración Base

La configuración principal vive en el archivo `akvorado.yaml`. Los cambios requieren reiniciar el orchestrator para aplicarse.

4.1 Puertos de recepción de flujos

Por defecto Akvorado escucha en los siguientes puertos UDP. Deben estar abiertos en el firewall:

Puerto	Protocolo
UDP 2055	NetFlow v5 / v9
UDP 4739	IPFIX
UDP 6343	sFlow

4.2 Retención de datos — Valores por defecto

Akvorado viene configurado con los siguientes valores de retención por defecto:

Resolución	TTL por defecto	Notas
0s (raw)	360h (15 días)	Alto consumo de disco con volúmenes grandes
1 minuto	168h (7 días)	Agregado por minuto
5 minutos	2160h (90 días)	Agregado por 5 minutos
1 hora	8760h (1 año)	Muy eficiente en disco

Importante: Los 15 días de retención en raw flows son el principal causante de saturación de disco en producción con volúmenes altos. Ver sección 5 para el ajuste aplicado.

4.3 Retención en Kafka — Valor por defecto

Por defecto Kafka retiene los mensajes durante 24 horas (86400000 ms). En entornos de alto volumen esto puede acumular demasiados datos si el outlet tiene alguna caída.

```
# Valor por defecto en akvorado.yaml
kafka:
  retention.ms: 86400000 # 24 horas
```

Info: Con un outlet estable, 6 horas de retención es más que suficiente como buffer. Ver sección 5 para el ajuste aplicado.

5. Ajustes Aplicados en Producción

Los siguientes ajustes se aplicaron para resolver problemas de saturación de disco y memoria que se presentaron en desarrollo con un volumen de ~75,000 a 103,000 flujos por segundo.

5.1 Resumen de cambios

Parámetro	Antes (default)	Después (ajustado)
TTL raw flows	360h (15 días)	24h (1 día)
Kafka retention	86400000 ms (24h)	21600000 ms (6h)
ClickHouse max memory	5 GB	8 GB
Max bytes to merge	3 GB	2 GB
Parts to delay insert	default	500
Parts to throw insert	default	600

5.2 Prerrequisito: Acceder a ClickHouse e identificar tablas

Antes de operar ClickHouse, es útil listar las tablas creadas por Akvorado para confirmar que todas las resoluciones estén presentes.

Comando:

```
SHOW TABLES FROM default;
```

Entre las tablas más importantes normalmente se encuentran:

```
flows          # la más pesada (raw flows)
flows_1m0s     # agregado por minuto
flows_5m0s     # agregado por 5 minutos
flows_1h0m0s   # agregado por hora (muy eficiente)
```

Siempre luego de modificar server.xml se puede recargar la configuración en caliente con SYSTEM RELOAD CONFIG, reiniciar únicamente el contenedor de ClickHouse, o bien bajar y levantar todo el stack con docker compose down && docker compose up -d si los cambios son parte de un ajuste más amplio. Reiniciar ClickHouse:

```
sudo docker compose -f /opt/akvorado/docker/docker-compose.yml restart clickhouse
```

Identificar el nombre del contenedor activo (puede variar según el nombre del proyecto Docker Compose):

```
sudo docker ps
```

Acceder al cliente interactivo de ClickHouse para validar configuraciones, merges, TTLs y estado general:

```
sudo docker exec -it docker-clickhouse-1 clickhouse-client
```


5.3 Problema: Disco lleno por retención excesiva

Problema detectado: El servidor llegó al 98% de uso de disco. ClickHouse había acumulado 144 GB de datos de flujos raw con 15 días de retención. El servicio dejó de funcionar y fue necesaria intervención de emergencia.

Solución aplicada: Se redujo el TTL de raw flows de 360h a 24h en la configuración de Akvorado. Se hizo TRUNCATE de la tabla flows para liberar el disco de inmediato, y se ajustó el TTL en la tabla directamente desde el cliente de ClickHouse.

Configuración corregida en akvorado.yaml:

```
clickhouse:
  resolutions:
    - interval: 0s
      ttl: 24h      # raw flows: 1 día
    - interval: 1m
      ttl: 168h     # por minuto: 7 días
    - interval: 5m
      ttl: 2160h    # por 5 min: 90 días
    - interval: 1h
      ttl: 8760h    # por hora: 1 año
```

Comando para truncar la tabla de emergencia (dentro del cliente de ClickHouse):

```
TRUNCATE TABLE flows SETTINGS max table size to drop=0;
```

Modificar TTL directamente en la tabla:

```
ALTER TABLE flows MODIFY TTL TimeReceived + toIntervalSecond(86400);
```

5.4 Problema: Errores de memoria en ClickHouse

Problema detectado: ClickHouse tenía un límite de 5 GB de RAM configurado. Al intentar hacer merges de particiones en background, el proceso excedía el límite y fallaba con MEMORY_LIMIT_EXCEEDED. Esto causaba que los flujos entrantes no se pudieran escribir correctamente y la UI mostrara datos incompletos o vacíos.

Solución aplicada: Se subió el límite de memoria de ClickHouse a 8 GB en el archivo clickhouse/server.xml, aprovechando que el servidor tiene 16 GB de RAM. Adicionalmente se redujo el tamaño máximo de merge para que cada operación consuma menos memoria.

Ajuste en clickhouse/server.xml:

```
<max_server_memory_usage>8589934592</max_server_memory_usage>
```

```
<merge_tree>
  <parts_to_delay_insert>500</parts_to_delay_insert>
  <parts_to_throw_insert>600</parts_to_throw_insert>

  <max_bytes_to_merge_at_max_space_in_pool>2147483648</max_bytes_to_merge_at_max_space_in_pool>
</merge_tree>
```

Para recargar la configuración sin reiniciar el servicio:

```
docker exec -it docker-clickhouse-1 clickhouse-client --query "SYSTEM RELOAD CONFIG"
```

5.5 Problema: Acumulación de partes sin mergear

Problema detectado: Con el disco lleno y los merges fallando, ClickHouse acumuló cientos de partes pequeñas sin consolidar en la tabla flows. Esto degradaba las queries y generaba más presión de memoria en cada intento de merge.

Solución aplicada: Al limpiar la tabla con TRUNCATE y reducir el TTL, la acumulación se detuvo. La configuración de parts_to_delay_insert y parts_to_throw_insert limita la velocidad de inserción cuando hay demasiadas partes, dando tiempo a ClickHouse para mergear.

Para monitorear el estado de las partes en cualquier momento:

```
SELECT partition, count() AS parts, formatReadableSize(sum(bytes_on_disk)) AS size
FROM system.parts WHERE table = 'flows' AND active = 1
GROUP BY partition ORDER BY partition DESC LIMIT 30;
```

5.6 Ajuste en Kafka

Problema detectado: Kafka tenía 24 horas de retención por defecto. Con el outlet caído durante períodos de mantenimiento, Kafka acumulaba un backlog grande que al recuperarse generaba picos de inserción masivos en ClickHouse.

Solución aplicada: Se redujo la retención de Kafka a 6 horas. Esto es suficiente como buffer para cubrir interrupciones breves del outlet sin saturar el sistema al recuperarse.

```
kafka:
  retention.ms: 21600000    # 6 horas
```

Recomendado: Para revisar si el outlet está atrasado en consumir Kafka: `docker exec -it docker-kafka-1 /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server kafka:9092 --describe --group akvorado-outlet`

5.7 Ajustes avanzados de TTL y limpieza automática

Para estabilizar el comportamiento de ClickHouse bajo cargas altas de NetFlow/sFlow también se aplicaron ajustes adicionales en `merge_tree` dentro de `clickhouse/server.xml`.

Configuración aplicada:

```
<ttl_only_drop_parts>1</ttl_only_drop_parts>
<merge_with_ttl_timeout>3600</merge_with_ttl_timeout>
<max_number_of_merges_with_ttl_in_pool>4</max_number_of_merges_with_ttl_in_pool>
```

Objetivo de los cambios:

- `ttl_only_drop_parts=1` permite eliminar particiones completas expiradas en lugar de borrar filas individualmente.
- `merge_with_ttl_timeout=3600` fuerza revisión de TTL cada 1 hora en vez del valor por defecto (~4 horas).
- `max_number_of_merges_with_ttl_in_pool=4` permite hasta 4 merges TTL simultáneos para acelerar limpieza de partes expiradas.

Resultado observado:

Reducción significativa de backlog de merges, liberación más rápida de espacio en disco y estabilización de la tabla flows durante cargas altas.

5.8 Ajuste de memoria recomendado (12 GB)

La configuración base establece 8 GB de RAM para ClickHouse. Si la VM dispone de 16 GB o más de RAM libre, se recomienda subir el límite a 12 GB para proporcionar mayor margen operativo durante merges intensivos, consultas históricas y crecimiento futuro de ingestión.

Archivo: `/opt/akvorado/docker/clickhouse/server.xml`

```
<!-- Subir de 8 GB a 12 GB si la VM tiene 16-18 GB RAM -->
<max_server_memory_usage>12884901888</max_server_memory_usage>
```

5.9 Validaciones operativas recomendadas

Luego de aplicar los ajustes de tuning se recomienda validar periódicamente el estado de ClickHouse.

Validar configuraciones activas:

```
SELECT name, value FROM system.merge_tree_settings WHERE name IN
('ttl_only_drop_parts','merge_with_ttl_timeout','max_number_of_merges_with_ttl_in_pool');
```

Validar cantidad de particiones activas:

```
SELECT partition, count() AS parts FROM system.parts WHERE active AND database='default'
AND table='flows' GROUP BY partition ORDER BY partition;
```

Validar tamaño lógico de la tabla flows:

```
SELECT formatReadableSize(sum(bytes_on_disk)) AS size, sum(rows) AS rows, count() AS
active_parts FROM system.parts WHERE active AND database='default' AND table='flows';
```

Validar uso de disco del host:

```
df -h
```

Validar uso de recursos de contenedores:

```
sudo docker stats --no-stream
```

5.10 Consideraciones operativas adicionales

Durante las pruebas en desarrollo se identificó que restores pesados de Proxmox podían generar latencia de I/O y gaps temporales en ingestión.

Recomendaciones:

- Evitar restores o snapshots pesados durante ventanas críticas de operación.
- Utilizar almacenamiento SSD dedicado cuando sea posible.
- Mantener monitoreo de Kafka para detectar backlog temprano.
- Documentar qué router exporta BMP y qué tablas/rutas está enviando.

5.11 Estado estable observado luego del tuning

Luego de aplicar todos los ajustes descritos anteriormente, el entorno quedó estable bajo carga sostenida.

Resultados observados:

- Retención estable de ~24 horas en raw flows
- 25–26 particiones activas sostenidas
- Uso de disco estabilizado (~57–62%)
- Ingestión sostenida cercana a 25k flows/s
- Sin eventos recurrentes de 'Too many parts'
- Limpieza TTL funcionando correctamente
- Kafka sin backlog significativo
- ClickHouse estable durante múltiples días consecutivos

6. Implementación de BMP

BMP (BGP Monitoring Protocol) es un protocolo que permite recibir información BGP directamente desde los routers en tiempo real, sin depender de SNMP ni de lookups externos.

6.1 ¿Qué aporta BMP a Akvorado?

Sin BMP, Akvorado enriquece los flujos con datos BGP de manera indirecta. Con BMP activado, el outlet recibe una sesión BGP/BMP directa desde el router y puede enriquecer cada flujo con:

Campo enriquecido	Descripción
DstASPath	AS path completo del prefijo de destino
Dst1stAS / Dst2ndAS / Dst3rdAS	Primeros ASNs del path — útil para identificar el proveedor inmediato
DstCommunities	Comunidades BGP del prefijo
DstLargeCommunities	Large communities BGP
SrcAS / DstAS	ASN de origen y destino con mayor precisión

Recomendado: Con BMP activo, los reportes de tráfico por ASN son mucho más precisos porque la información BGP viene directamente del router en lugar de ser inferida.

6.2 Configuración en el Outlet

El outlet es el componente que recibe las sesiones BMP. Se habilita escuchando en el puerto TCP 10179 (puerto estándar de BMP):

En akvorado.yaml, bajo el bloque outlet:

```
outlet:
  bmp:
    listen: 0.0.0.0:10179
```

En el docker-compose.yml, el puerto debe estar expuesto:

```
services:
  akvorado-outlet:
    ports:
      - "10179:10179/tcp"
```

6.3 Configuración en el Router

En el router se debe configurar una sesión BMP apuntando al servidor Akvorado. El router envía actualizaciones BGP al servidor cada vez que hay un cambio en la tabla de ruteo.

Importante: La configuración específica del router depende del fabricante (Juniper, Cisco, Arista, etc.). Consultar la documentación del equipo para habilitar BMP hacia una dirección IP externa.

6.4 Verificación

Una vez configurado, verificar que el outlet reciba la sesión BMP:

```
docker logs docker-akvorado-outlet-1 --tail 50 | grep -i bmp
```

También en la consola web, los flujos deberían mostrar el AS path completo en los campos DstASPath y los campos Dst1stAS / Dst2ndAS / Dst3rdAS con valores reales.

7. Configuración de Exportadores

Esta sección cubre cómo configurar los equipos de red para enviar datos a Akvorado. Hay dos mecanismos principales: sFlow (para switches Capa 2/3) y BMP (para routers BGP). Adicionalmente se cubre SNMP para enrichment de interfaces.

7.1 sFlow — Switches Capa 2/3

sFlow es el mecanismo para exportar muestras de tráfico desde switches de acceso/distribución. Opera en Capa 2/3 y permite visibilidad de tráfico incluso sin routing BGP. El switch del ejemplo a continuación es un equipo Arista/EOS.

Configuración en switch (ejemplo IXP-SW7 con EOS/Arista):

```
sflow sample 1024
sflow polling-interval 30
sflow destination 192.168.20.8      # IP del servidor Akvorado
sflow source-interface Vlan20      # Interfaz fuente del agente sFlow
```

Parámetros clave:

- sample 1024: 1 paquete muestreado de cada 1024. Para enlaces de 100G se puede bajar a 512. Para enlaces lentos subir a 2048 o más.
- polling-interval 30: estadísticas de interfaces cada 30 segundos.
- Puerto UDP destino: 6343 (valor por defecto de Akvorado para sFlow).

Para verificar que los flows llegan al servidor desde el switch:

```
sudo tcpdump -i any udp port 6343 -n -c 20
```

7.2 NetFlow v9 / IPFIX — Routers Capa 3

NetFlow e IPFIX son protocolos de exportación de flujos a nivel Capa 3, utilizados principalmente en routers. Son más precisos que sFlow porque exportan todos los flujos (no muestras). IPFIX es la evolución estandarizada de NetFlow v9.

Ejemplo de configuración NetFlow en Juniper (IOS-XR o JunOS similar):

```
flow-server {
    192.168.20.8 port 4739; # IPFIX
}
```

Puertos UDP de Akvorado para cada protocolo: NetFlow v5/v9 → 2055, IPFIX → 4739, sFlow → 6343. Todos deben estar abiertos en el firewall del servidor.

7.3 BMP (BGP Monitoring Protocol) — Router de Borde

BMP opera sobre TCP y es exclusivo de routers con sesiones BGP activas (Capa 3 / plano de control). A diferencia de sFlow/NetFlow que reportan tráfico del plano de datos, BMP envía actualizaciones de la tabla de ruteo BGP en tiempo real, lo que permite a Akvorado enriquecer los flows con AS path, comunidades y prefijos con máxima precisión.

Configuración en Juniper JunOS (router IXP-RT1) — producción CRIX:

```

set routing-options bmp station AKVORADO hold-down 30
set routing-options bmp station AKVORADO hold-down flaps 10
set routing-options bmp station AKVORADO hold-down period 30
set routing-options bmp station AKVORADO local-address 192.168.20.9 # IP del router
set routing-options bmp station AKVORADO connection-mode active
set routing-options bmp station AKVORADO monitor enable
set routing-options bmp station AKVORADO route-monitoring pre-policy # tabla antes
de filtros
set routing-options bmp station AKVORADO route-monitoring post-policy # tabla después
de filtros
set routing-options bmp station AKVORADO station-address 192.168.20.8 # IP del
servidor Akvorado
set routing-options bmp station AKVORADO station-port 10179 # puerto TCP
BMP
set routing-options bmp station AKVORADO statistics-timeout 60

```

Descripción de los parámetros clave:

- connection-mode active: el router inicia la conexión TCP hacia Akvorado (recomendado).
- route-monitoring pre-policy: exporta rutas tal como llegan antes de aplicar políticas de importación.
- route-monitoring post-policy: exporta rutas que quedan activas después de los filtros. Recomendado habilitar ambos para máxima visibilidad.
- hold-down: evita reconexiones agresivas si la sesión BMP falla repetidamente.
- statistics-timeout 60: el router envía estadísticas BMP cada 60 segundos.

Verificar sesión BMP activa desde el router Juniper:

```

show bmp
show bmp station detail

```

Verificar desde el servidor Akvorado que el puerto BMP está escuchando y recibiendo conexiones:

```

ss -tlnp | grep 10179 # puerto TCP escuchando
docker logs docker-akvorado-outlet-1 --tail 50 | grep -i bmp

```

7.4 SNMP — Enriquecimiento de Interfaces

Akvorado usa SNMP para consultar los exportadores y obtener los nombres de interfaces (ifName), descripciones (ifDescr) y velocidades. Sin SNMP, las interfaces aparecen sólo como números (ifIndex). Es el mecanismo más sencillo de enriquecimiento y funciona tanto con switches como con routers.

Configuración en akvorado.yaml — bloque inlet:

```

inlet:
  snmp:
    community: public # comunidad SNMP v2c
    cache-duration: 30m # cache de resultados SNMP
    timeout: 5s
    retries: 3

```

Configuración de SNMP en el exportador (ejemplo switch Arista/EOS):

```

snmp-server community public ro
snmp-server host 192.168.20.8 version 2c public # notificaciones hacia Akvorado

```

Configuración SNMP en router Juniper:

```
set snmp community public authorization read-only
set snmp client-list AKVORADO 192.168.20.8/32
```

Verificar que Akvorado puede hacer SNMP al exportador desde el servidor:

```
snmpwalk -v2c -c public 192.168.20.9 ifDescr 2>/dev/null | head -10
```

7.5 Resumen: ¿Qué protocolo usar según el equipo?

Tipo de Equipo	Protocolo flows	BMP	SNMP
Switch L2/L3 (Arista, Cisco, etc.)	sFlow	No aplica	Sí — nombres de interfaces
Router de borde BGP (Juniper, Cisco)	NetFlow v9 / IPFIX	Sí — enriquecimiento BGP completo	Sí
Route-server BIRD (IXP Manager)	No exporta flows	No recomendado (config auto)	No aplica

8. Monitoreo y Operaciones Diarias

Esta sección cubre los comandos del día a día para mantener Akvorado saludable, monitorear ingestión, revisar logs y validar el estado de ClickHouse, Kafka y los contenedores.

8.1 Estado general del sistema

Verificar que todos los contenedores están corriendo y sanos:

```
sudo docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Consumo de CPU, RAM y red por contenedor (snapshot en tiempo real):

```
sudo docker stats --no-stream
```

Uso de disco del host:

```
df -h
```

Carga del servidor (CPU, RAM, procesos):

```
htop
```

8.2 Monitoreo de logs

Logs del inlet (recepción de flows):

```
sudo docker logs docker-akvorado-inlet-1 --tail 100 -f
```

Logs del outlet (consumo Kafka + escritura ClickHouse):

```
sudo docker logs docker-akvorado-outlet-1 --tail 100 -f
```

Logs de ClickHouse (errores de memoria, merges, TTL):

```
sudo docker logs docker-clickhouse-1 --tail 200 -f
```

Buscar errores críticos en logs de ClickHouse:

```
sudo docker logs docker-clickhouse-1 2>&1 | grep -iE "error|exception|memory|parts" | tail -50
```

Logs de Kafka:

```
sudo docker logs docker-kafka-1 --tail 100 -f
```

8.3 Monitoreo de Kafka

Verificar backlog del consumer group del outlet (lag = cuántos mensajes está atrasado):

```
sudo docker exec -it docker-kafka-1 /opt/kafka/bin/kafka-consumer-groups.sh \
--bootstrap-server kafka:9092 --describe --group akvorado-outlet
```

Listar topics de Kafka:

```
sudo docker exec -it docker-kafka-1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 --list
```

Ver detalles y offsets de un topic:

```
sudo docker exec -it docker-kafka-1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 --describe --topic flows
```

8.4 Monitoreo de ClickHouse

Acceder al cliente ClickHouse:

```
sudo docker exec -it docker-clickhouse-1 clickhouse-client
```

Queries de monitoreo dentro de ClickHouse:

```
-- Uso de disco por tabla
SELECT table, formatReadableSize(sum(bytes_on_disk)) AS size, sum(rows) AS rows FROM
system.parts WHERE active GROUP BY table ORDER BY sum(bytes_on_disk) DESC;
-- Merges activos en progreso
SELECT database, table, elapsed, progress, is_mutation FROM system.merges ORDER BY
elapsed DESC;
-- Insercciones recientes (tasa de ingestión)
SELECT event_time, table, rows, read_bytes FROM system.query_log WHERE
type='QueryFinish' AND query ILIKE '%INSERT%' ORDER BY event_time DESC LIMIT 20;
-- Memoria usada actualmente
SELECT formatReadableSize(value) AS memory_used FROM system.asynchronous_metrics WHERE
metric = 'MemoryResident';
-- Queries en ejecución en este momento
SELECT query_id, elapsed, memory_usage, query FROM system.processes ORDER BY elapsed
DESC;
```

8.5 Operaciones de reinicio y mantenimiento

Reiniciar un contenedor específico:

```
sudo docker compose -f /opt/akvorado/docker/docker-compose.yml restart clickhouse
sudo docker compose -f /opt/akvorado/docker/docker-compose.yml restart outlet
sudo docker compose -f /opt/akvorado/docker/docker-compose.yml restart inlet
```

Reiniciar todo el stack:

```
cd /opt/akvorado/docker && sudo docker compose down && sudo docker compose up -d
```

Recargar configuración de ClickHouse sin reiniciar (para cambios en server.xml que soporten hot-reload):

```
sudo docker exec -it docker-clickhouse-1 clickhouse-client --query "SYSTEM RELOAD
CONFIG"
```

Actualizar imagen de un servicio (sin afectar datos):

```
cd /opt/akvorado/docker && sudo docker compose pull && sudo docker compose up -d
```

8.6 Verificar tasa de ingestión en tiempo real

Verificar flows llegando al puerto sFlow (UDP 6343):

```
sudo tcpdump -i any udp port 6343 -n -c 20
```

Verificar flows NetFlow/IPFIX (puertos 2055 y 4739):

```
sudo tcpdump -i any udp port 2055 or udp port 4739 -n -c 20
```

Flows por segundo en ClickHouse (últimas 5 particiones):

```
SELECT partition, formatReadableSize(sum(bytes_on_disk)), sum(rows) FROM system.parts
WHERE table='flows' AND active GROUP BY partition ORDER BY partition DESC LIMIT 5;
```

8.7 Demo y referencia oficial de Akvorado

Akvorado ofrece un entorno de demostración público con datos reales donde se puede explorar la interfaz, consultas y funcionalidades antes de tener el propio sistema configurado. Es útil para validar que la UI propia se comporta igual que la referencia, y para mostrar capacidades a terceros.

Demo oficial: <https://demo.akvorado.net>

En el demo se puede explorar: dashboards de tráfico por ASN, país, interfaz y protocolo; consultas históricas; y el comportamiento de BMP con datos BGP enriquecidos.

9. Troubleshooting

Guía rápida de diagnóstico para los problemas más comunes en producción con Akvorado y ClickHouse.

9.1 No llegan flows / UI sin datos

Síntoma: La consola web no muestra tráfico o muestra datos muy antiguos.

Diagnóstico paso a paso:

1. Verificar que el exportador está enviando al puerto correcto del servidor Akvorado.

```
sudo tcpdump -i any udp port 6343 or udp port 2055 or udp port 4739 -n -c 30
```

2. Revisar que el contenedor inlet está corriendo y procesando:

```
sudo docker logs docker-akvorado-inlet-1 --tail 50 | grep -iE "error|flows|received"
```

3. Verificar que Kafka tiene mensajes en el topic:

```
sudo docker exec -it docker-kafka-1 /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server kafka:9092 --describe --group akvorado-outlet
```

4. Revisar que el outlet está escribiendo a ClickHouse:

```
sudo docker logs docker-akvorado-outlet-1 --tail 50 | grep -iE "error|clickhouse|insert"
```

9.2 Disco lleno — intervención de emergencia

Síntoma: `df -h` muestra +90% en la partición de ClickHouse. El servicio puede dejar de escribir.

Acciones inmediatas (dentro del cliente clickhouse-client):

```
-- 1. Ver cuánto ocupa cada tabla
SELECT table, formatReadableSize(sum(bytes_on_disk)) FROM system.parts WHERE active
GROUP BY table ORDER BY 2 DESC;
-- 2. TRUNCATE de emergencia (borra todo y libera espacio inmediatamente)
TRUNCATE TABLE flows SETTINGS max_table_size_to_drop=0;
-- 3. Ajustar TTL directamente en la tabla (sin reiniciar)
ALTER TABLE flows MODIFY TTL TimeReceived + toIntervalSecond(86400);
```

9.3 Error: Too many parts

Síntoma: ClickHouse rechaza inserciones con "Too many parts". Generalmente el conteo de parts activos supera 600 en la tabla flows.

Diagnóstico:

```
SELECT partition, count() AS parts FROM system.parts WHERE active AND table='flows'
GROUP BY partition ORDER BY parts DESC LIMIT 10;
```

Solución: Forzar merge en la partición con más parts (reemplazar YYYYMMDD HH con la partición afectada):

```
OPTIMIZE TABLE flows PARTITION '2025-05-27 10' FINAL;
```

Si persiste, verificar que `parts_to_delay_insert` y `parts_to_throw_insert` están configurados (ver sección 5.3).

9.4 Error: MEMORY_LIMIT_EXCEEDED en ClickHouse

Síntoma: Logs muestran "MEMORY_LIMIT_EXCEEDED". Los merges fallan y la UI puede mostrar datos incompletos.

Verificar uso de memoria actual:

```
SELECT formatReadableSize(value) FROM system.asynchronous_metrics WHERE  
metric='MemoryResident';
```

Solución: Incrementar max_server_memory_usage en server.xml (ver secciones 5.3 y 5.12) y reiniciar ClickHouse. Si la VM no tiene RAM disponible, reducir max_bytes_to_merge_at_max_space_in_pool para que los merges sean más pequeños.

9.5 BMP no conecta / flows sin enriquecimiento BGP

Síntoma: Los flows llegan pero los campos DstASPath, Dst1stAS y comunidades aparecen vacíos.

Diagnóstico:

```
ss -tlnp | grep 10179 # verificar que el puerto está escuchando  
docker logs docker-akvorado-outlet-1 --tail 100 | grep -i bmp  
sudo tcpdump -i any tcp port 10179 -n -c 20 # paquetes BMP llegando?
```

Causas frecuentes: el puerto 10179 no está expuesto en docker-compose.yml, el firewall del servidor bloquea TCP 10179, la IP de station-address en el router no coincide con la IP del servidor Akvorado, o el router no tiene configuración de connection-mode active.

9.6 Interfaces aparecen como números en la UI

Síntoma: Las interfaces se muestran como "ifIndex 512" en lugar del nombre real (ej. "Ethernet1/1").

Causa: SNMP no está configurado en Akvorado o el exportador no responde. Diagnóstico:

```
snmpwalk -v2c -c public <IP-DEL-EXPORTADOR> ifDescr 2>>&1 | head -10
```

Si el comando devuelve error, verificar que la comunidad SNMP en akvorado.yaml coincide con la configurada en el exportador, y que el firewall permite UDP 161 desde el servidor Akvorado hacia el exportador.

9.7 Kafka con backlog alto al reiniciar el outlet

Síntoma: Al reiniciar el outlet tras un período de caída, ClickHouse recibe un pico masivo de inserciones y puede llegar a "Too many parts" o errores de memoria.

Mitigación: Si el backlog es mayor a 1 hora y no es crítico recuperarlo, se puede resetear el offset del consumer group para que el outlet empiece desde el final (se pierden los flows acumulados pero se estabiliza el sistema):

```
-- Primero detener el outlet  
sudo docker compose -f /opt/akvorado/docker/docker-compose.yml stop outlet  
-- Resetear offset al final del topic (se descartan mensajes acumulados)  
sudo docker exec -it docker-kafka-1 /opt/kafka/bin/kafka-consumer-groups.sh --  
bootstrap-server kafka:9092 --group akvorado-outlet --topic flows --reset-offsets --  
to-latest --execute  
-- Reiniciar outlet  
sudo docker compose -f /opt/akvorado/docker/docker-compose.yml start outlet
```

9.8 Tabla de referencia rápida

Síntoma	Causa probable	Acción
UI sin datos	Exportador no envía, outlet caído	tcpdump + docker logs outlet
Disco +90%	TTL raw flows muy alto	TRUNCATE + reducir TTL
Too many parts	Merges insuficientes, parts acumulados	OPTIMIZE TABLE + ajustes 5.6
MEMORY_LIMIT_EXCEEDED	Límite RAM bajo para merges	Subir max_server_memory_usage
Sin enriquecimiento BGP	BMP no conecta, puerto no expuesto	ss -tlnp + logs outlet grep bmp
Interfaces como números	SNMP no configurado o bloqueado	snmpwalk desde servidor a exportador
Pico de inserciones al reiniciar	Backlog en Kafka acumulado	Reset offset Kafka a --to-latest